

Osztályozás

Bevezető

Bevezetés az osztályozásba

Mi az osztályozás szerepe a gépi tanulásban?

- Az osztályozás egy *felügyelt tanulási* feladat, amelynek célja az adatok adott (zárt) kategóriákba sorolása.

Gyakori alkalmazási területek?

- Széles körben alkalmazzák az orvosi diagnosztikában, pénzügyi elemzésekben, szövegbányászatban stb.

Fajtái?

- Bináris (két kategória) vagy multiosztályos (több kategória) osztályozás.

Konfúziós mátrix

- Alapvetően egy táblázat, amely az osztályozási modell teljesítményét mutatja be az egyes osztályok helyes és helytelen előrejelzései alapján.

Jósolt érték

Valódi értékek			
		Pozitív	Negatív
Jósolt érték	Pozitív	TP	FP
	Negatív	FN	TN

Konfúziós mátrix

Alapvető elemei:

- Igaz pozitív (**TP**): Azok az esetek, amelyeket helyesen soroltak be a pozitív osztályba.
- Hamis pozitív (**FP**): Azok az esetek, amelyeket tévesen soroltak be a pozitív osztályba.
- Igaz negatív (**TN**): Azok az esetek, amelyeket helyesen soroltak be a negatív osztályba.
- Hamis negatív (**FN**): Azok az esetek, amelyeket tévesen soroltak be a negatív osztályba.

Jósolt érték

Valódi értékek			
		Pozitív	Negatív
Jósolt érték	Pozitív	TP	FP
	Negatív	FN	TN

Konfúziós mátrix

Felhasználása és értelmezése:

- A konfúziós mátrix alapján különböző *teljesítménymutatók* számolhatók, mint például a pontosság, precizitás, visszahívás és az F1-mutató.
- Segít az osztályozási hibák *megértésében* és az algoritmusok finomhangolásában.
- *Kiegyensúlyozatlan adatok* esetén különösen fontos az egyes osztályokhoz tartozó hibák vizsgálata.

Jósolt érték

Valódi értékek			
		Pozitív	Negatív
Jósolt érték	Pozitív	TP	FP
	Negatív	FN	TN

Pontosság - *accuracy*

Mérőszám, amely megmutatja, hogy a klasszifikáló algoritmus hány esetben osztályozta helyesen az adatokat.

- Képlete:

$$accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Jósolt érték

Valódi értékek			
		Pozitív	Negatív
Jósolt érték	Pozitív	TP	FP
	Negatív	FN	TN

Precizitás - *precision*

Megmutatja, hogy a klasszifikáló hány esetben osztályozta helyesen a pozitív osztályt a pozitív osztályozások közül.

- Képlete:

$$precision = \frac{TP}{TP+FP}$$

(pl. spam szűrés, jogi vizsgálatok)

Jósolt érték

Pozitív
Negatív

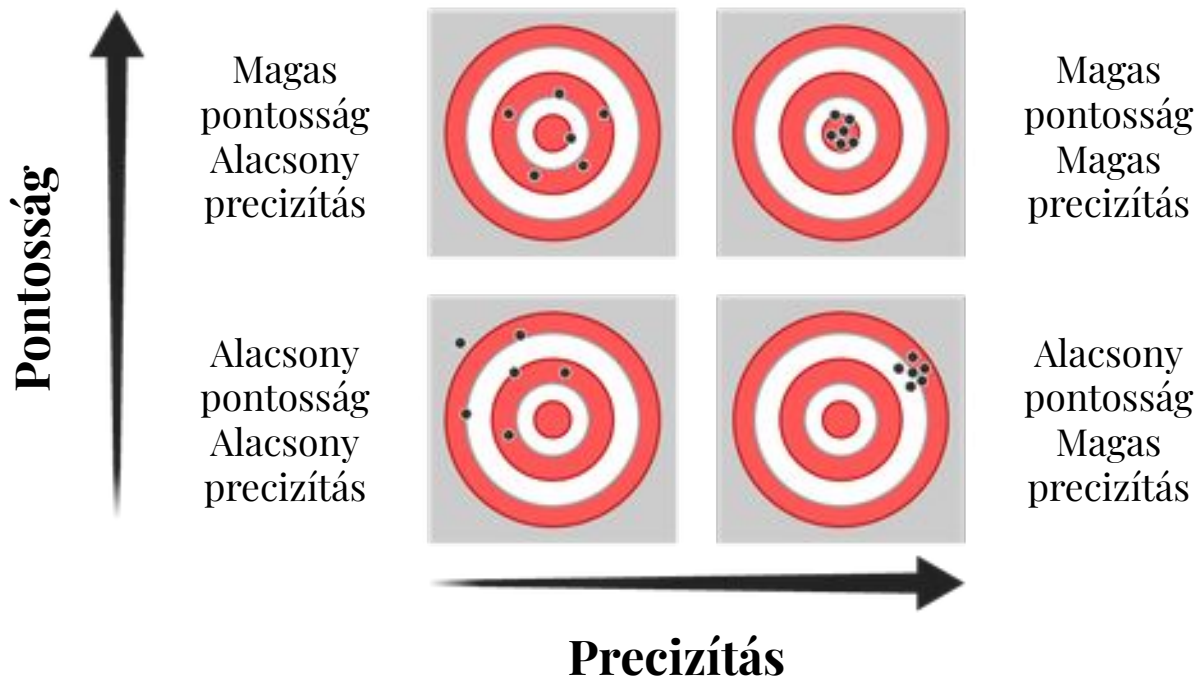
Valódi értékek

Pozitív

Negatív

Jósolt érték	Valódi értékek	
	Pozitív	Negatív
Pozitív	TP	FP
Negatív	FN	TN

Pontosság vs. precizitás



Kiegyensúlyozatlan adatok esetén

Miért nem elég az *accuracy* kiegyensúlyozatlan adatok esetén?

- azt méri, hogy a modell előrejelzéseinek hány százaléka helyes. Ez azonban kiegyensúlyozatlan osztályok esetén félrevezető lehet, mert a ritkábban előforduló osztályokat nem veszi megfelelően figyelembe.

Például

- 990 egészséges beteg (negatív osztály - "nem beteg")
- 10 beteg személy (pozitív osztály - "beteg")

Kiegyensúlyozatlan adatok esetén

Például

- Osztályozónk minden betegre azt mondja majd, hogy egészséges!
- Bár az *accuracy* magas, a modell teljesen használhatatlan, mert egyik beteget sem ismerte fel.

	Jósolt nem beteg	Jósolt beteg
Nem beteg	990	0
Beteg	10	0

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} = \frac{0 + 990}{0 + 990 + 0 + 10} = \frac{990}{1000} = 99\%$$

Szenzitivitás - *recall* // *true positive rate*

Megmutatja, hogy a klasszifikáló hány esetben osztályozta helyesen a pozitív osztályt a tényleges pozitív esetek közül.

- Képlete:

$$recall = \frac{TP}{TP+FN}$$

(pl. rákdiagnosztika, biztonsági riasztások)

Jósolt érték

Valódi értékek			
		Pozitív	Negatív
Jósolt érték	Pozitív	TP	FP
	Negatív	FN	TN

F1-score

A precizitás és a visszahívás harmonikus átlaga, amely egyensúlyt teremt a két mérőszám között.

- Képlete:

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

(pl. ritka osztályok)

...ezen kívül még:

- Specificitás // true negative rate
- Hamis pozitív arány
- Hamis negatív arány
- Dice-score (DSC)
- False discovery rate
- False omission rate
- ROC-görbe alatti terület
- stb.

		Predicted condition		
		Predicted positive	Predicted negative	
Total population = P + N				Informedness, bookmaker informedness (BM) $= \text{TPR} + \text{TNR} - 1$ Prevalence threshold (PT) $= \frac{\sqrt{\text{TPR} \times \text{FPR}} - \text{FPR}}{\text{TPR} - \text{FPR}}$
Actual condition	Real Positive (P) ^[a]	True positive (TP), hit ^[b] False negative (FN), miss, underestimation	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{\text{TP}}{\text{P}} = 1 - \text{FNR}$	False negative rate (FNR), miss rate, type II error ^[c] $= \frac{\text{FN}}{\text{P}} = 1 - \text{TPR}$
	Real Negative (N) ^[d]	False positive (FP), false alarm, overestimation True negative (TN), correct rejection ^[e]	False positive rate (FPR), probability of false alarm, fall-out, type I error ^[f] $= \frac{\text{FP}}{\text{N}} = 1 - \text{TNR}$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{\text{TN}}{\text{N}} = 1 - \text{FPR}$
Prevalence $= \frac{\text{P}}{\text{P} + \text{N}}$	Positive predictive value (PPV), precision $= \frac{\text{TP}}{\text{TP} + \text{FP}} = 1 - \text{FDR}$	False omission rate (FOR) $= \frac{\text{FN}}{\text{TN} + \text{FN}} = 1 - \text{NPV}$	Positive likelihood ratio (LR+) $= \frac{\text{TPR}}{\text{FPR}}$	Negative likelihood ratio (LR-) $= \frac{\text{FNR}}{\text{TNR}}$
Accuracy (ACC) $= \frac{\text{TP} + \text{TN}}{\text{P} + \text{N}}$	False discovery rate (FDR) $= \frac{\text{FP}}{\text{TP} + \text{FP}} = 1 - \text{PPV}$	Negative predictive value (NPV) $= \frac{\text{TN}}{\text{TN} + \text{FN}} = 1 - \text{FOR}$	Markedness (MK), deltaP (Δp) $= \text{PPV} + \text{NPV} - 1$	Diagnostic odds ratio (DOR) $= \frac{\text{LR}+}{\text{LR}-} = \frac{\text{TP} \times \text{TN}}{\text{FP} \times \text{FN}}$
Balanced accuracy (BA) $= \frac{\text{TPR} + \text{TNR}}{2}$	F ₁ score $= \frac{2 \text{PPV} \times \text{TPR}}{\text{PPV} + \text{TPR}} = \frac{2 \text{TP}}{2 \text{TP} + \text{FP} + \text{FN}}$	Fowkes–Mallows index (FM) $= \sqrt{\text{PPV} \times \text{TPR}}$	phi or Matthews correlation coefficient (MCC) $= \frac{\sqrt{\text{TPR} \times \text{TNR} \times \text{PPV} \times \text{NPV}}}{\sqrt{\text{FNR} \times \text{FPR} \times \text{FOR} \times \text{FDR}}}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{\text{TP}}{\text{TP} + \text{FN} + \text{FP}}$

További metrikák

Python: adatok előkészítése

```
array = dataset.values      # adathalmaz
X = array[:,0:4]            # az első 4 oszlop tartalmazza az attribútumokat
Y = array[:,4]              # 5. oszlop az osztálycímét tartalmazza
validation_size = 0.20      # validációs adat mérete (20%)

X_train,
X_validation,
Y_train,
Y_validation =
    model_selection.train_test_split(X, Y, test_size=validation_size, random_state=42)
```

Python: osztályozás

```
model = DecisionTreeClassifier()           # modell példányosítása
model.fit(X_train, Y_train)                # adatokra való illesztés
predictions = model.predict(X_validation)   # előrejelzés a tesztadatokon

# kiértékelés például:
# accuracy-ra
accuracy_score(Y_validation, predictions)

# konfúziós mátrix
confusion_matrix(Y_validation, predictions)

# klasszifikációs jelentés
classification_report(Y_validation, predictions)
```

Mire a 80-20% felosztás, miért a validációs adathalmaz?

80-20%

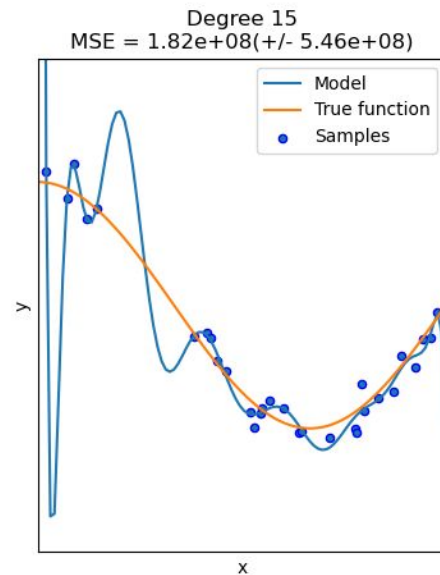
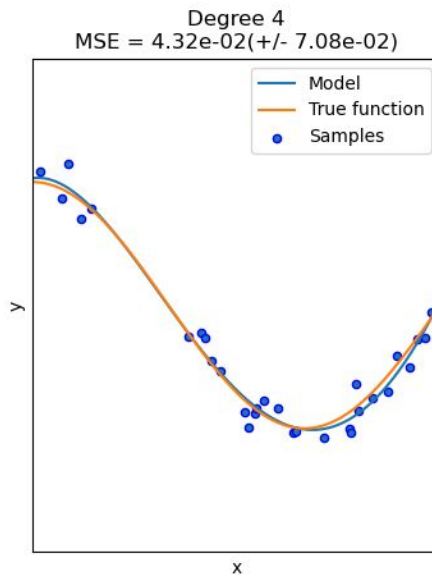
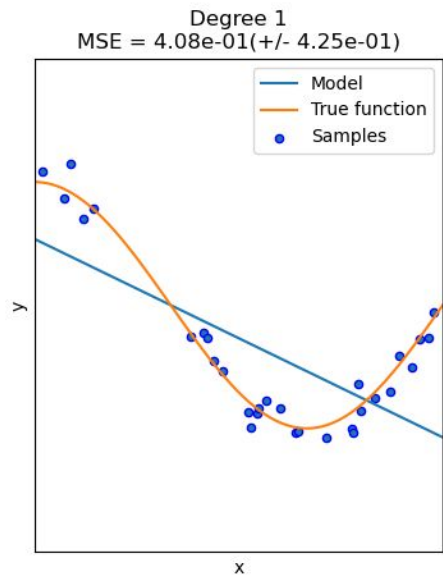
- Az adatok nagy részét a modell betanítására ($\mathcal{X}_{train}$, $\mathcal{Y}_{train}$), míg a többit validációra ($\mathcal{X}_{validation}$, $\mathcal{Y}_{validation}$) használjuk.
- Lehetne ez más érték is akár...

Miért?

- A validációs adathalmaz célja, hogy a modellt olyan adatokon teszteljük, amelyeket nem látott a tanulás során.
- Ez segít felmérni a modell általánosító képességét (illetve így össze lehet őket hasonlítani), illetve észrevenni a túlilleszkedést (*overfitting*).

Overfitting = túlzott alkalmazkodás a tanuló adatokhoz $\Rightarrow$ rossz teljesítmény új adatokon.

Underfitting vs. overfitting



https://scikit-learn.org/stable/auto_examples/model_selection/plot_underfitting_overfitting.html

Cross-validation

A cross-validation (keresztvalidáció) egy olyan módszer, amely segít a modell ne tanulja meg túl jól/ne overfitteljen a tesztadatokon.

A leggyakoribb keresztvalidációs módszer a *k-fold cross-validation*, amely a következőképpen működik:

1. Az adatokat k részre (*fold*) osztjuk.
2. Minden egyes részletet egyszer tesztadatként használunk, miközben a többi részt tréningadatokként.
3. A folyamatot k -szor ismételjük, minden egyes *fold*-nál más és más tesztadatot választunk.
4. Az összes eredmény átlagolása adja a modell általános teljesítményét

Pyhton: cross-validation

```
# inicializáljuk a modellt
model = DecisionTreeClassifier()

# 5-fold cross-validation elvégzése
cv_scores = cross_val_score(model, X, y, cv=5)

# eredmények
print("Cross-validation értékek:", cv_scores)
print("Átlag cross-validation érték:", cv_scores.mean())
```

*cross-validation használatakor nem kell az adathalmazt külön tanuló és tesztelő halmazokra osztani. A teljes a adathalmazt használja, de úgy, hogy minden adatot pontosan egyszer használ tesztadatként.